

Lab 7.2 Database & Visualization 2 - Practicing with Grafana Dashboard

2.1 Practice #1 Time series

Let's add panels to your dashboard. We will practice three different types of panels (see the dashboard 'Lab7/Lab7 sample'): 1) Time series as Figure 7d, 2) Table as Figure 7a, and 3) Discrete as Figure 7c. To create Time series panel for DHT11 temperature data visualization, please follow the steps below.

1. Click 'add panel' icon on the top menu bar.
2. Click 'Add a new panel' and then you will see the panel editing page as Figure 14. Each section of the page is described below.
 - a. Preview section: You can see the preview of the panel you are editing.
 - b. Query section: You can customize query to request data to the database
 - c. Property section: You can manipulate the appearances of the panel

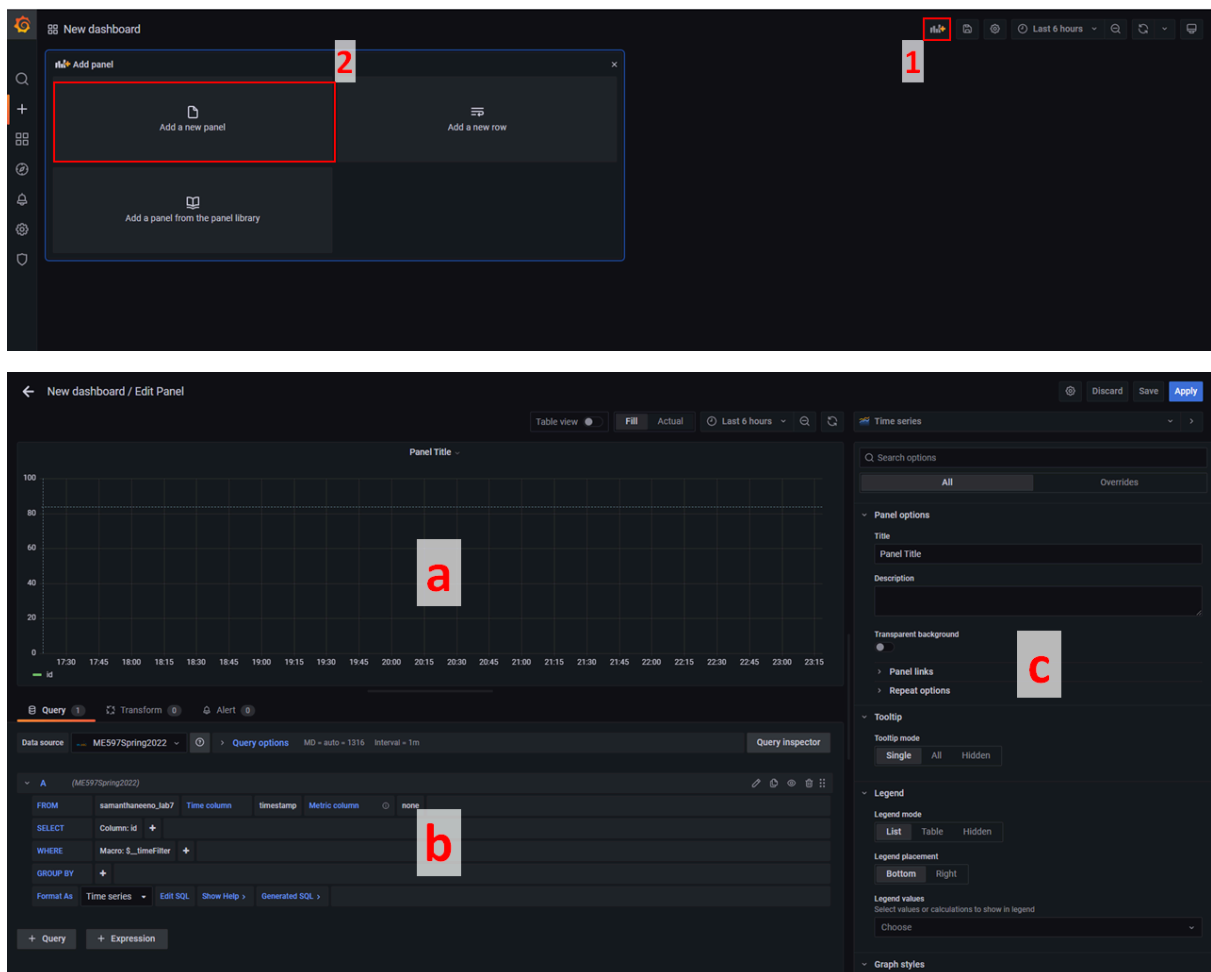
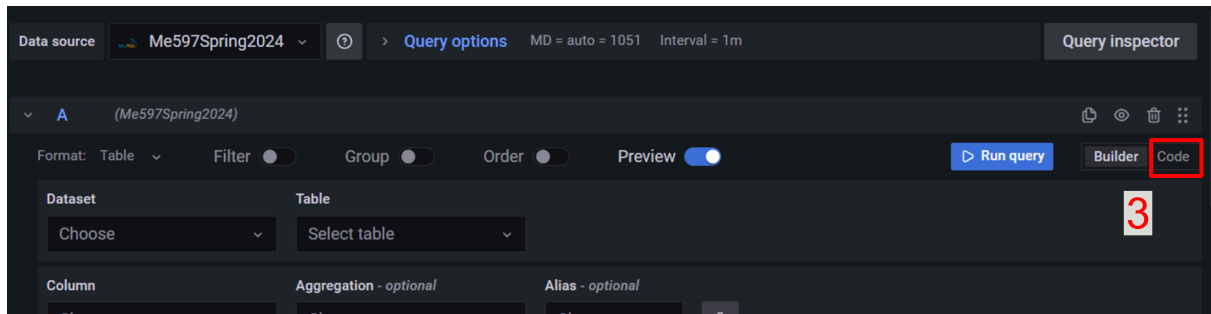


Figure 14 Panel editing page

3. Click the 'Code' button in the Query section to switch to the SQL query statement

a. Please note that in this lab, you do not need to change 'Data source' because the '**ME597Spring24**' MySQL database is set by the default data source of this Grafana server. However, when you need to use a different data source, ask it the administrator of the Grafana server.



4. Write down the SELECT statement below to get 'timestamp' and 'value' to visualize temperature data according to time on the Query section.

SQL - SELECT statement for Practice #1

```
SELECT
  timestamp AS "time",
  0+value as "Temperature"
FROM grafana_sample
WHERE
  $__timeFilter(timestamp) AND
  measurement = "temp" AND
  value != "UNAVAILABLE"
ORDER BY timestamp
```

Tip - SELECT statement in Grafana

1. The reason why put '0+' in front of 'value' column in the SELECT statement is to make the data in a numeric data type. We set up the 'value' column in the MySQL as 'TINYTEXT' that means basically return data is string data type.
2. In addition, in the WHERE clause, 'value != "UNAVAILABLE"' makes your SELECT statement exclude "UNAVAILABLE" data in the 'value' column.
3. '\$__' is indicating the macro function in Grafana to correlate the time range of Grafana interface with the database. '\$__timeFilter(timestamp)' means timestamps between the given time range. If you click 'Show Help' in the Query section, you will see the details of descriptions about the macro as below.

```

Format As Time series Query Builder Show Help v Generated SQL >

Time series:
- return column named time or time_sec (in UTC), as a unix time stamp or any sql native date data type. You can use the macros below.
- return column(s) with numeric datatype as values
Optional:
- return column named metric to represent the series name.
- If multiple value columns are returned the metric column is used as prefix.
- If no column named metric is found the column name of the value column is used as series name

Resultsets of time series queries need to be sorted by time.

Table:
- return any set of columns

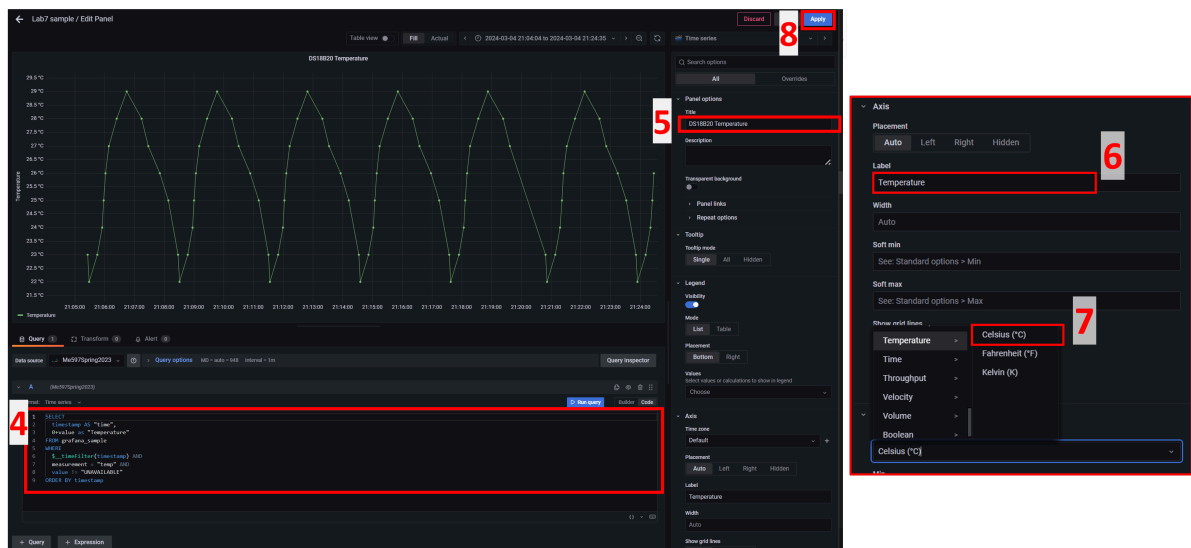
Macros:
- $__time(column) -> UNIX_TIMESTAMP(column) as time_sec
- $__timeepoch(column) -> UNIX_TIMESTAMP(column) as time_sec
- $__timefilter(column) -> column BETWEEN FROM_UNIXTIME(1492758877) AND FROM_UNIXTIME(1492758877)
- $__timeepochfilter(column) -> time_unix_epoch > 1492758877 AND time_unix_epoch < 1492758877
- $__timeepochnanofilter(column) -> column >= 1494418783152415214 AND column <= 1494497183142514872
- $__timegroup(column, '5m') -> cast(cast(UNIX_TIMESTAMP(column))/(300) as signed)*300 as signed
  by setting fillvalue grafana will fill in missing values according to the interval
  fillvalue can be either a literal value, NULL or previous; previous will fill in the previous seen value or NULL if none has been seen yet
- $__timegroupalias(column, '5m') -> cast(cast(UNIX_TIMESTAMP(column))/(300) as signed)*300 as signed AS "time"
- $__unixepochgroup(column, '5m') -> column DIV 300 * 300
- $__unixepochgroupalias(column, '5m') -> column DIV 300 * 300 AS "time"

Example of group by and order by with $__timeGroup:
SELECT
  $__timeGroupAlias(timestamp_col, '1h'),
  sum(value_double) as value
FROM yourtable
GROUP BY 1
ORDER BY 1

Or build your own conditionals using these macros which just return the values:
- $__timeFrom() -> FROM_UNIXTIME(1492758877)
- $__timeTo() -> FROM_UNIXTIME(1492758877)
- $__unixepochFrom() -> 1492758877
- $__unixepochTo() -> 1492758877
- $__unixepochNanoFrom() -> 1494418783152415214
- $__unixepochNanoTo() -> 1494497183142514872

```

5. Make 'Title' as 'DS18B20 Temperature' on the Property section
 6. Make 'Label' as 'Temperature' on the Property section
 7. Select 'Unit' as 'Celsius (°C)' on the Property section
- a. You will find 'Unit' and 'Label' properties if scroll down on the Property section. 8. Click 'Apply' on the right side on the top menu.



You will see the created panel for DHT11 temperature data as Figure 15. **Please note that if you want to add more data in the same graph panel as Figure 7f, click '+Query' on the bottom of the Query section.**

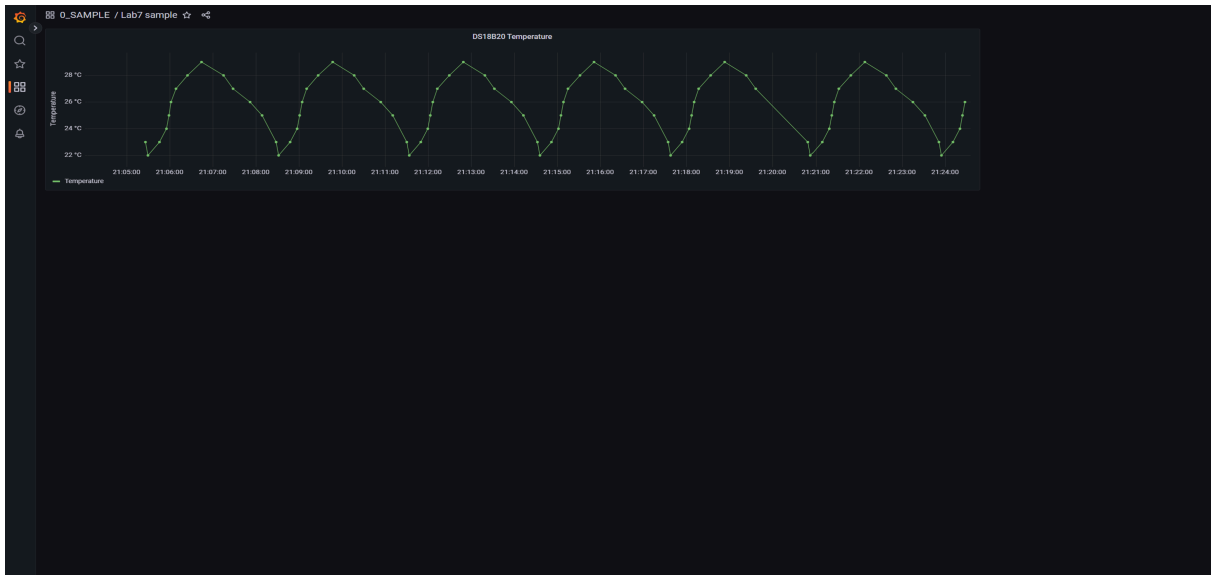


Figure 15 Panel added 1: DHT11 temperature

2.2 Practice #2 Table

Let's add Table panel for ADXL345 sensor data in table format as Figure 7a. Follow the steps below.

1. Click 'add panel' icon on the top menu bar and then click 'Add a new panel' as you did in the previous part
2. In the panel editing page, click 'Click to change visualization'
3. Click 'Table'
4. Click 'Toggle text edit mode' icon in the Query section to switch to the SQL query statement

5. Write down the SELECT statement below to get data to visualize the ADXL345 sensor data in the table format on the Query section.

a. In this case, because table can show both string and numeric data, you do not need to put '+0' in front of the 'value' column in SELECT statement.


```

SELECT
    timestamp AS "time",
    sensor,
    measurement,
    value
FROM grafana_sample
WHERE
    $__timeFilter(timestamp) AND
    sensor = "ADXL345"
ORDER BY timestamp

```

6. Make 'Title' as 'ADXL345 sensor data'

7. Click 'Apply'

You will see the created Table panel for ADXL345 sensor data as Figure 16. Please note that the Table panel is good to show EVENT data such as alarm, condition, state and so on.

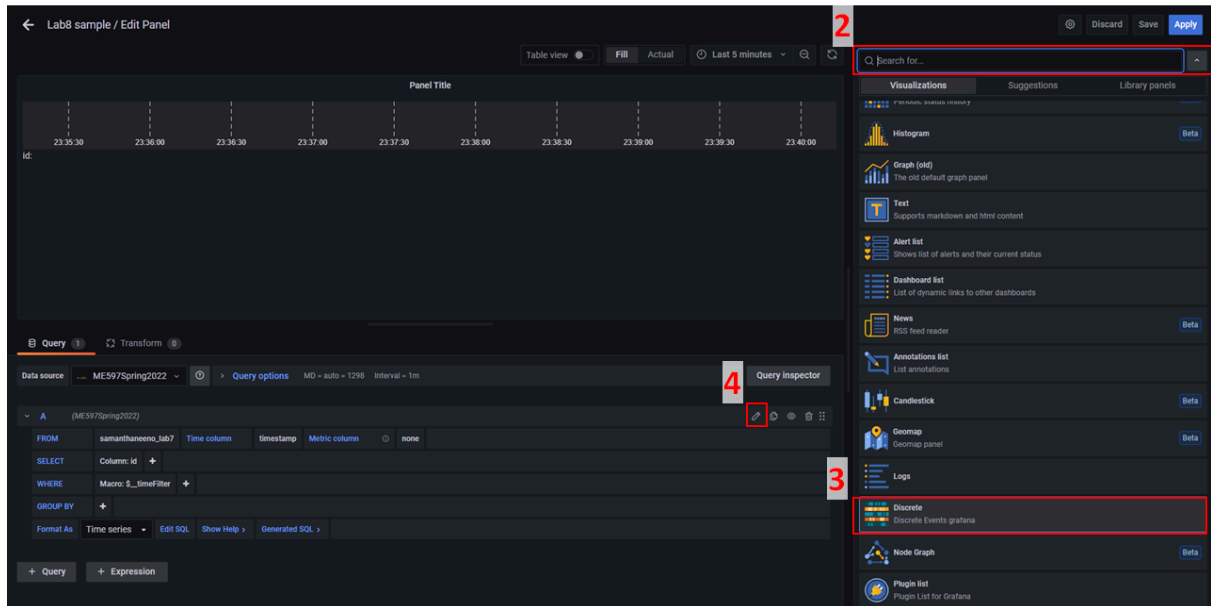


Figure 16 Panel added 2: ADXL345 sensor data table

2.3 Practice #3 Table

Let's add Discrete panel as Figure 7c. The Discrete panel allows us to see the discrete Event data in a direct and straightforward way. To create Discrete panel for power state data, please follow the steps below.

1. Click 'add panel' icon on the top menu bar and then click 'Add a new panel' as you did in the previous part
2. In the panel editing page, click 'Click to change visualization'
3. Click 'Discrete'
4. Click 'Toggle text edit mode' icon in the Query section to switch to the SQL query statement

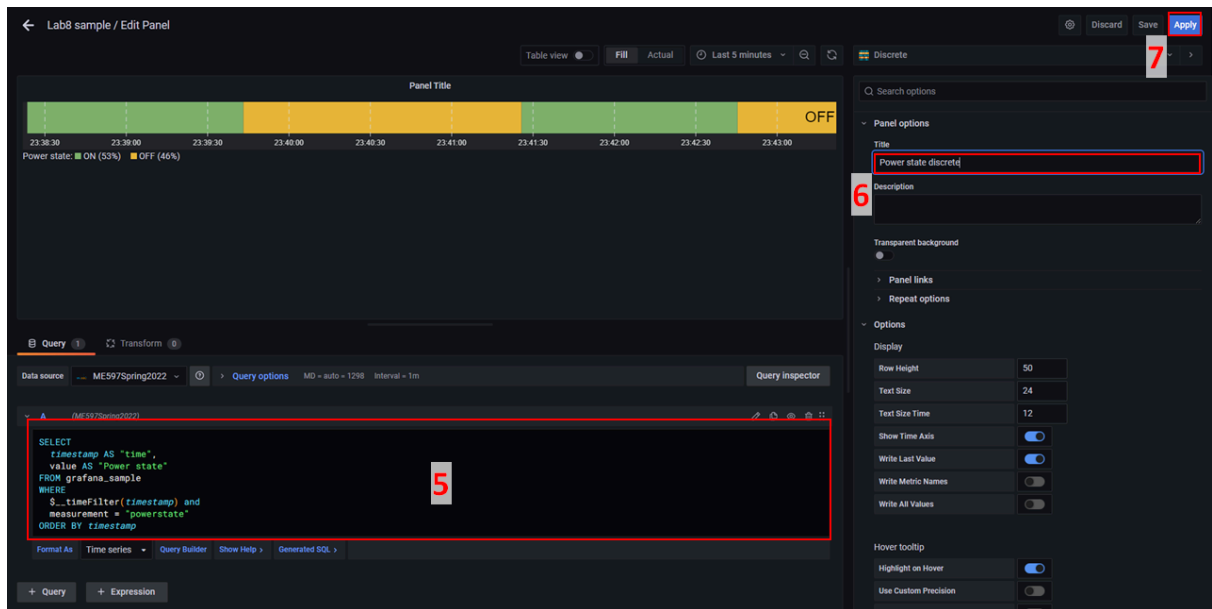


5. Write down the SELECT statement below to get discrete Event data to visualize the power state of vacuum pump on the Query section.

SQL - SELECT statement for Practice #3

```
SELECT
  timestamp AS "time",
  value AS "Power state"
FROM grafana_sample
WHERE
  $__timeFilter(timestamp) AND
  measurement = "powerstate"
ORDER BY timestamp
```

6. Make 'Title' as 'Power state dashboard'
7. Click 'Apply'



You will see the created Discrete panel for power state Event data as Figure 17. Also, you can adjust each panel size by using mouse. To change the size of the panel, put your mouse pointer on the corner of the panel. To move the panel, put your mouse pointer on the title, and then drag to another placement as Figure 18. **Please note that after every update of your dashboard, please save it!**

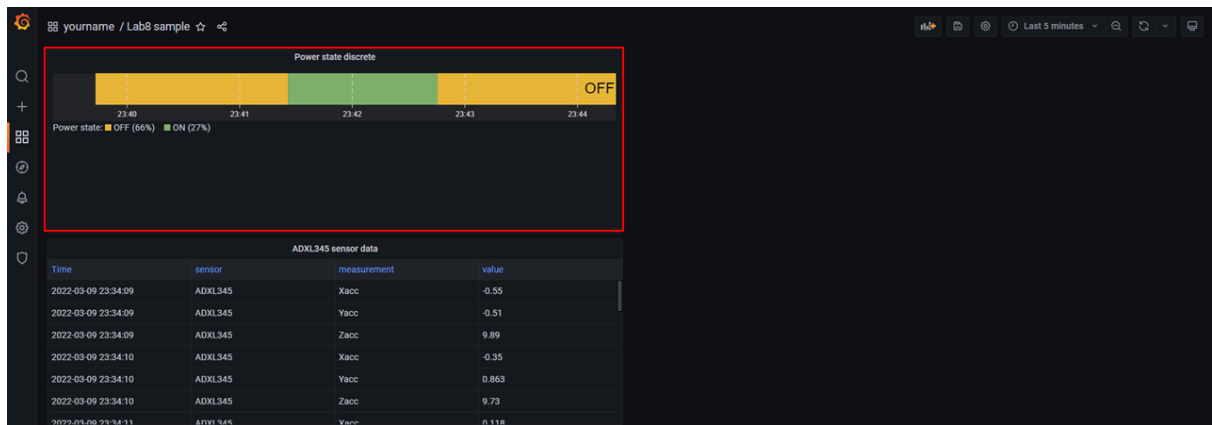


Figure 17 Panel added 3: Power state discrete Event data

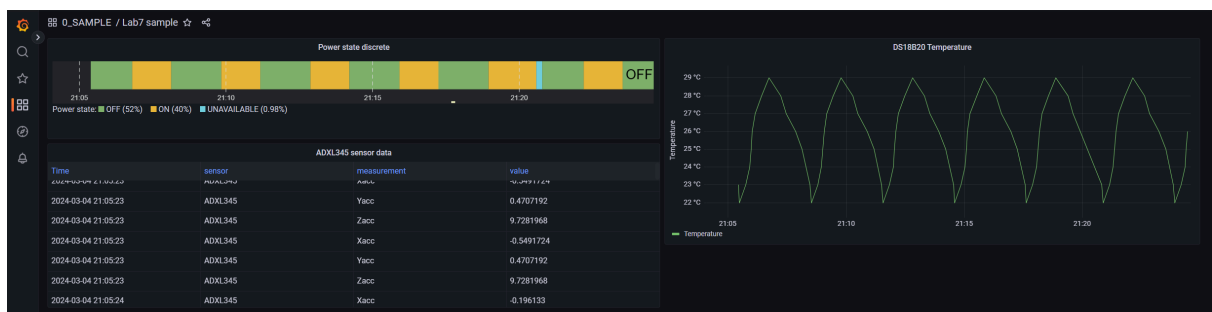
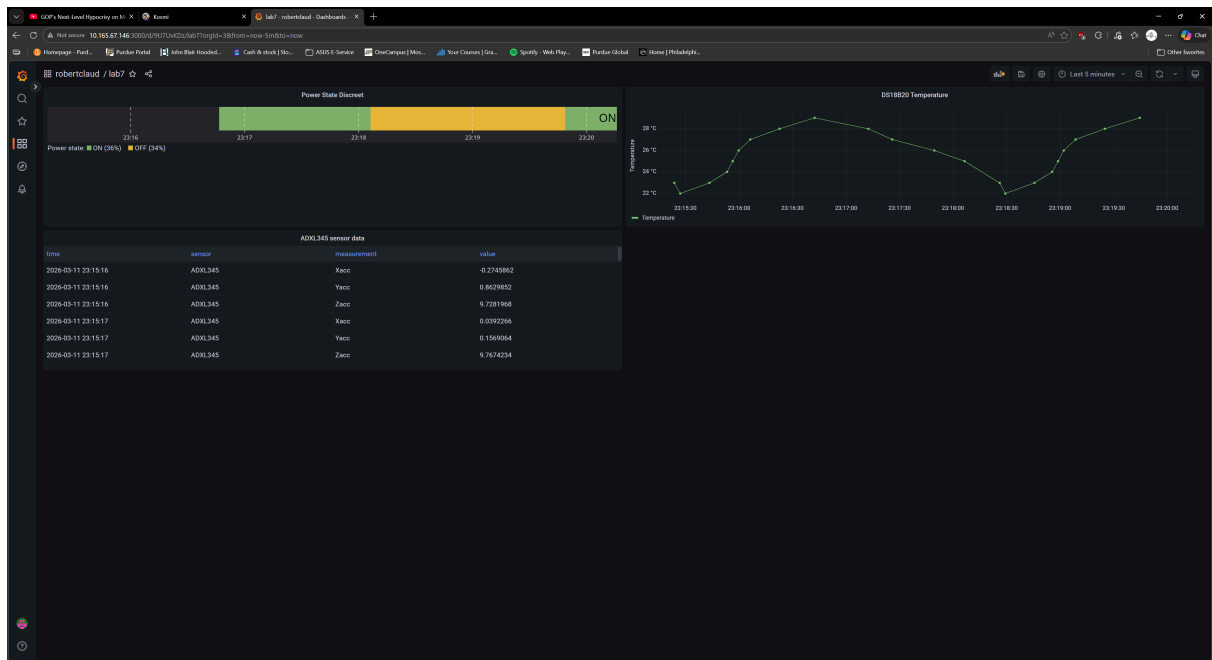


Figure 18 Added panels after adjusting each panel size and the placement.

Task 2.1

After performing the above parts, capture your created dashboard ('Practice') as Figure 18 and then attach it to the report. Do not remove the dashboard even after finishing this task. Please make sure that your dashboard should be in your folder.



2.4 Build Up The Entire Monitoring System

Finally, in this section, you need to build up the entire system as Figure 19. Perform the last task below. MTConnect collector Python script (the answer of Prelab7 Task1.4) using XML parsing is given. But you have to modify some front parts indicated with asteriks below.

Python - Python3 (mtconnect_collector.py)

```
## Credential
* HOST = 'MySQL HOST DNS' # MySQL server host DNS
PORT = 3306 # MySQL server port number
* USER = 'your account' # MySQL account name
* PASSWORD = 'your password' # Password of the account
* DB = 'Database information' # DB name
* TABLE = 'your table name' # table name
## Credential

## MTConnect info.
* agent = "agent host"
* agent_port = "agent port number"
url_current = "http://" + agent + ":" + agent_port + "/current"
MTCONNECT_STR = ET.fromstring(requests.get(url_current).content).tag.split("}")[0]+"}"
print("MTConnect string header is {}".format(MTCONNECT_STR))
## MTConnect info.
```

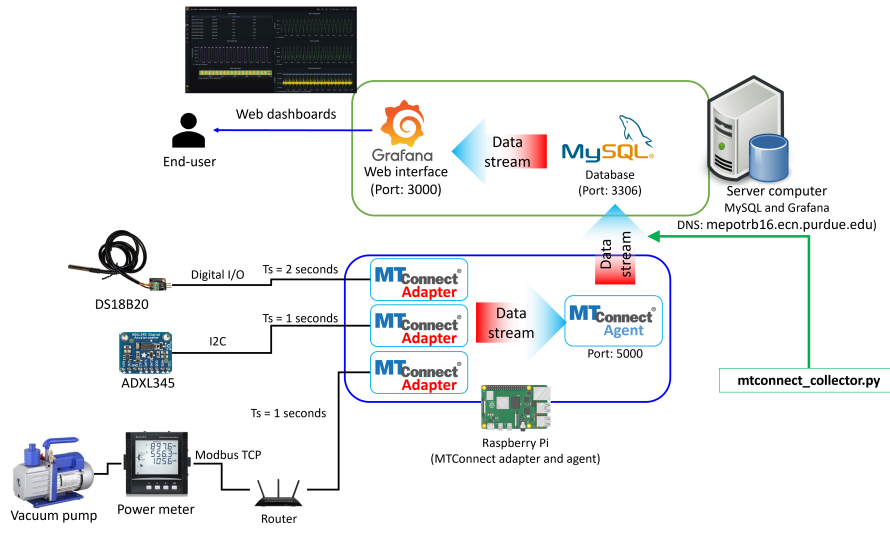


Figure 19 Schematic of the entire monitoring system for Lab7

Table Name: grafana_sample		Schema: ME697Spring24	
Charset/Collation: utf8mb4	utf8mb4_0900_ai_ci	Engine: InnoDB	
Comments:			
Column Name	Datatype	PK	NN
id	INT	☒	☒
timestamp	TIMESTAMP(6)	☐	☒
sensor	VARCHAR(45)	☐	☒
measurement	VARCHAR(45)	☐	☒
value	TINYTEXT	☐	☒

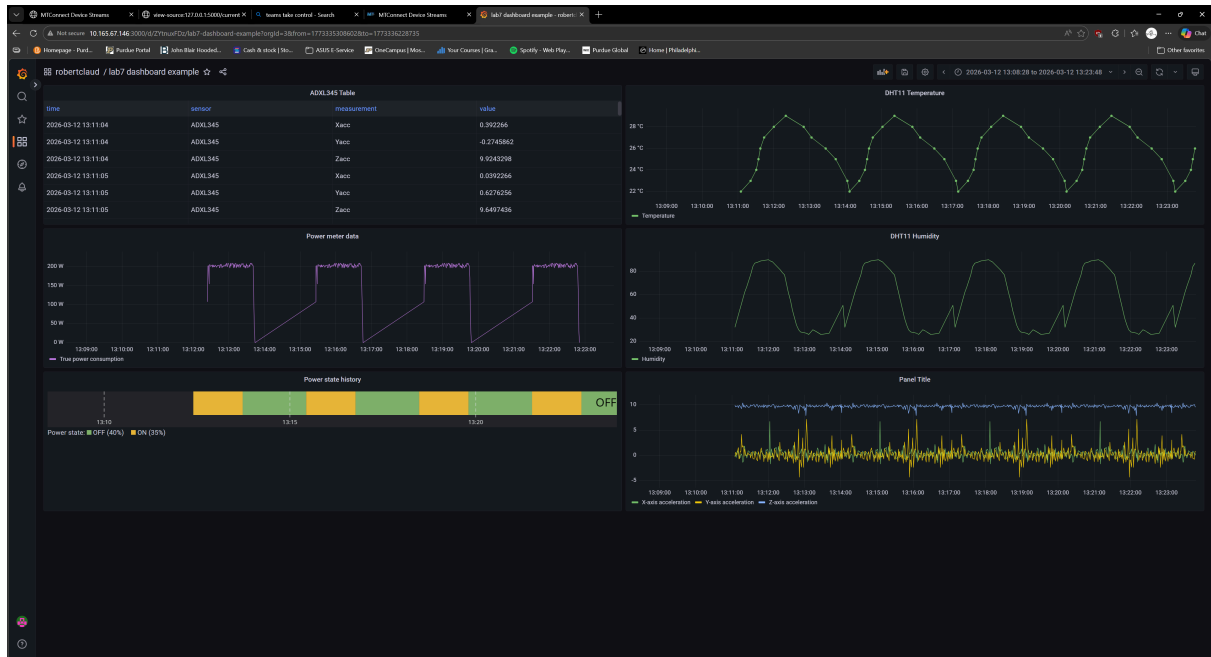
Figure 20 TIMESTAMP datatype update (TIMESTAMP(6))

Task 2.2

- Build up the entire monitoring system as Figure 19. The requirements are below.
 - You need to measure all three-sensor data (DS18B20, ADXL345, and power meter): This is the same as you did in Lab5.
 - DS18B20: temperature (Sample)
 - Virtual sensor: humidity (Sample) - use 'random' module in python language
 - ADXL345: X-axis acceleration (Sample)
 - ADXL345: Y-axis acceleration (Sample)
 - ADXL345: Z-axis acceleration (Sample)
 - Power meter: True power consumption (Sample)
 - Power meter: Power state (Event) - 'ON' or 'OFF' by a logic based on the measured true power consumption
 - The sample period of each sensor is 2, 1, and 1 second, respectively.
 - Follow MTConnect standard as much as you can.
 - Use the MySQL database table you created in Lab6 ('yourname_lab6'). * However please update your 'timestamp' column property to have 6 decimal precision of 'TIMESTAMP' data type as Figure 20.
 - Create a Grafana dashboard as the title of 'Task 2.2' in your name folder
 - Make panels in the created dashboard as Figure 7 ('LAB7/LAB7 dashboard example')
- By manipulating the sensors, confirm your dashboard shows data change.
 - Hint1: You can make 'device.xml' based on '<http://mepotrb16.ecn.purdue.edu:5000/probe>'
 - Hint2: Recall to the activities of Lab5.3 (MTConnect adapter).

- Hint3: It does not matter on which computer (Raspberry Pi or laptop) you run the 'mtconnect_collector.py'. But please confirm again the MySQL credentials and the MTConnect agent information.

4. Capture the created dashboard as [Figure 7](#) (Grafana:0_SAMPLE/Lab7 dashboard example) and then attach it to the report.



5. Copy and past contents of both 'device.xml' and 'agent.cfg' in the text blocks below.

```
<MTConnectDevices xmlns:m="urn:mtconnect.org:MTConnectDevices:1.1"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns="urn:mtconnect.org:MTConnectDevices:1.1"
  xsi:schemaLocation="urn:mtconnect.org:MTConnectDevices:1.1
http://www.mtconnect.org/schemas/MTConnectDevices_1.1.xsd">
  <Header creationTime="2010-03-04T18:44:40+00:00" sender="localhost" instanceId="1267728234"
bufferSize="131072" version="1.1"/>
  <Devices>
    <Device id="adx1345" name="ADXL345" uuid="001">
      <DataItems>
        <DataItem category="EVENT" id="001" name="avail" type="AVAILABILITY"/>
        <DataItem category="SAMPLE" id="a1" name="Xacc" type="ACCELERATION"
units="MILLIMETER/SECOND2"/>
        <DataItem category="SAMPLE" id="a2" name="Yacc" type="ACCELERATION"
units="MILLIMETER/SECOND2"/>
        <DataItem category="SAMPLE" id="a3" name="Zacc" type="ACCELERATION"
units="MILLIMETER/SECOND2"/>
      </DataItems>
    </Device>
    <Device id="dht11" name="DHT11" uuid="002">
      <DataItems>
        <DataItem category="EVENT" id="002" name="avail" type="AVAILABILITY"/>
        <DataItem category="SAMPLE" id="h1" name="humd" type="HUMIDITY_RELATIVE" subType="ACTUAL"
units="PERCENT"/>
        <DataItem category="SAMPLE" id="t1" name="temp" type="TEMPERATURE" subType="ACTUAL"
units="CELIUS"/>
      </DataItems>
    </Device>
  </Devices>
</MTConnectDevices>
```

```

    </Device>
    <Device id="power_meter" name="POWER_METER" uuid="003">
      <DataItems>
        <DataItem category="EVENT" id="003" name="avail" type="AVAILABILITY"/>
        <DataItem category="SAMPLE" id="p1" name="power" type="WATTAGE" subType="ACTUAL"
units="WATT"/>
        <DataItem category="EVENT" id="ps" name="powerstate" type="POWER_STATE" subType="LINE"/>
      </DataItems>
    </Device>
  </Devices>
</MTConnectDevices>

```

```

Devices = ./Device.xml
Port = 5000
AllowPut = true
ReconnectInterval = 1000
BufferSize = 17
MonitorConfigFiles = true
Pretty = true
SchemaVersion = 1.5

```

```

Adapters {
  # Log file has all machines with device name prefixed
  Device{
    Device = ADXL345
    Host = 192.168.50.190
    Port = 7878
  }
  Device1 {
    Device = DHT11
    Host = 192.168.50.190
    Port = 7879
  }
  Device2 {
    Device = POWER_METER
    Host = 192.168.50.190
    Port = 7880
  }
}

```

```

Files {
  schemas {
    Path = ./schemas
    Location = /schemas/
  }
  styles {
    Path = ./styles
    Location = /styles/
  }
  Favicon {
    Path = ./styles/favicon.ico
    Location = /favicon.ico
  }
}

```

```

StreamsStyle {
  Location = /styles/Streams.xml
}

```

```

# Logger Configuration
logger_config
{
  logging_level = debug

```

```
    output = cout  
}
```

Task 2.3

1. Is it possible to build the IIoT monitoring system without MTConnect? Discuss the pros and cons of using MTConnect in the system.
-

It is possible to build the IIoT monitoring system without MTConnect, although this would require a different communication protocol to unify the sensor readings, or building separate parsers for each sensor in the system.

Pros

- Unifies all sensor communication into a single protocol
- Outputs sensor readings in easily parsed XML

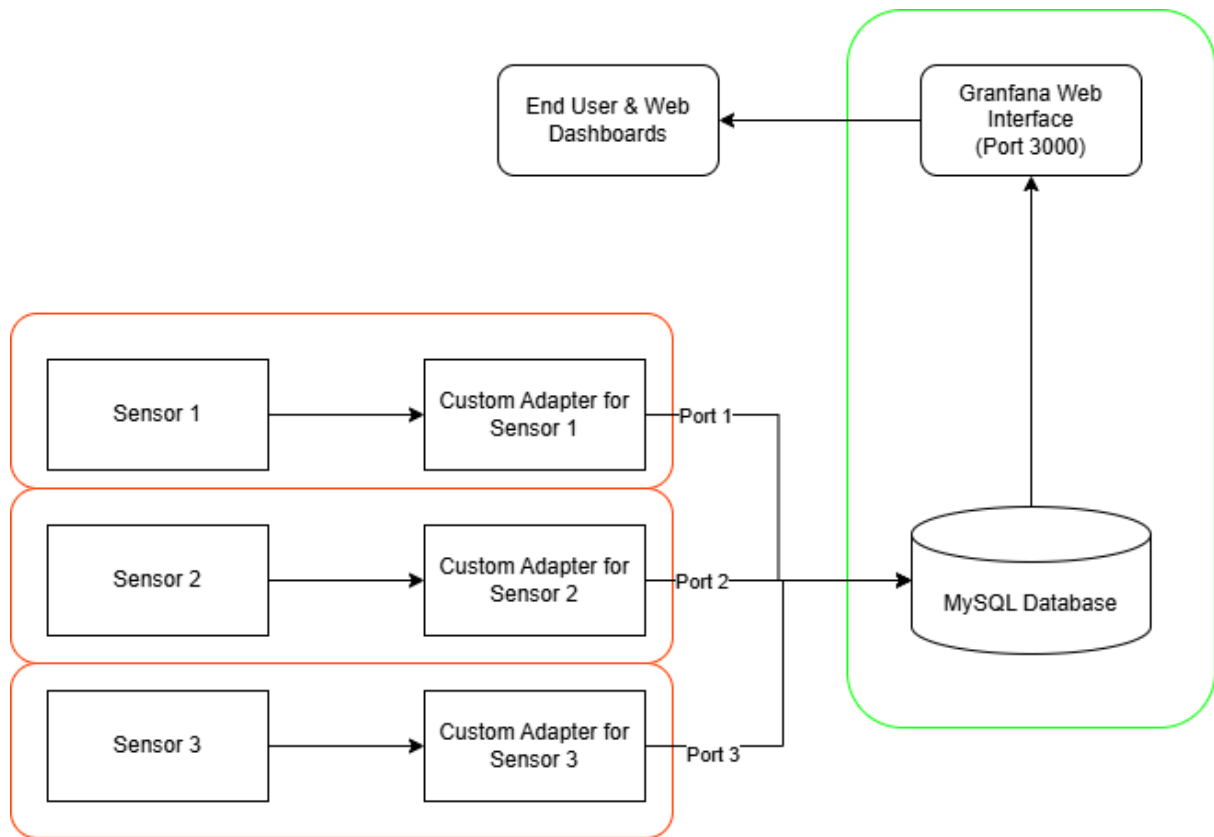
Cons

- Requires MTConnect compatible devices
 - Requires additional equipment to interpret sensor readings (Raspberry PI in this case)
-

2. Can you design the system without MTConnect, and how? Justify your strategies.

* You don't need to answer this question in detail. Answer this question in terms of the system configuration and the software you use without MTConnect. You can use figures, diagrams, etc.

As mentioned above, it would be possible to design the system without MTConnect using custom built adapters for each sensor. This would increase the overhead needed to design the system, but it would allow sensor readings to be inserted directly into the database without needing to wait for the latency of an agent or adapter. As shown below, a system could be designed with three custom adapters tailored for each sensor. This data could then be dumped directly into the MySQL database.



Lab7 Summary and Deliverables

Answer the following questions for your achievements

Q1. Please summarize Lab7.

In Lab 7 we learned how to create IoT dashboards using the open source software Grafana. We learned how to incorporate the learnings from previous labs where we connected several sensors together using the Raspberry Pi using MTConnect from Lab 5, and then dumped this data into the MySQL database that we set up in Lab 6. Then, using the data collected we built a dashboard to track the readings from all of the sensor outputs.

Q2. What skills did you have to develop to accomplish this project?

I had to continue to develop my networking skills to successfully connect all of the aspects of the system together - the sensors, the Raspberry Pi, the MTConnect agent and adapters, the MySQL database, and the newly introduced Grafana dashboard system.

Q3. What aspects of this project were the most beneficial for your learning?

The aspects of the project that were the most beneficial for my learning was that it incorporated all of the aspects of the prior labs together. I really appreciated seeing how everything culminated into a single project to show how the data can flow from the sensor all the way up to a digestible user interface.

Q4. What challenges did you encounter in completing the project?

For this project, I was unable to successfully get the MTConnect agent to parse the output data from my Raspberry Pi. I was able to take readings when I changed the XML to incorporate all sensor readings into the same sensor, but when I broke up the XML to distinguish between sensors it showed that they were unavailable.

Q5. How did you overcome the challenges or remedy the problems encountered?

To remedy the challenge, I first consulted ChatGPT which suggested that I change my adapter settings to send differently named "availability" events. This unfortunately did not work. I then called the TA to see if he could help but he was unable to diagnose the issue. Finally, I ended up using the MTConnect simulator set up on port 5000 of the class DNS.

Get back to [Lab Index Page](#)